Développement Web Avancé

Framework Laravel 11

Chapitre 2 : Routage et Contrôleurs

Enseignant
M. JARRAY Mondher

Classe
MP1ILC

Semestre
S1

Durée
1h30

Objectifs du Chapitre

- ✓ Comprendre le système de routage de Laravel
- ✓ Créer des routes avec différentes méthodes HTTP
- ✓ Utiliser les paramètres de route
- ✓ Nommer les routes pour une meilleure organisation
- ✓ Créer et utiliser des contrôleurs
- ✓ Organiser les routes avec des groupes et des middlewares

Plan du Chapitre

1. **Introduction au Routage**
   - Qu'est-ce qu'une route ?
   - Les fichiers de routes
2. **Les Routes de Base**
   - Méthodes HTTP (GET, POST, PUT, DELETE)
   - Routes avec paramètres
3. **Routes Nommées et Groupes**
   - Nommer les routes
   - Préfixes et groupes
4. **Les Contrôleurs**
   - Création de contrôleurs
   - Contrôleurs de ressources
5. **Les Middlewares**

1. Qu'est-ce qu'une Route ?

Le point d'entrée de votre application

Définition

Une **route** est une règle qui dit à Laravel : "Quand un utilisateur visite cette URL, exécute ce code".

Comment ça fonctionne ?

Utilisateur
Tape une URL

→

Routeur
Trouve la route

→

Contrôleur
Traite la demande

→

Réponse
Page HTML

Exemple simple

Quand l'utilisateur visite `http://monsite.com/bonjour`

```php
// routes/web.php
Route::get('/bonjour', function () {
    return 'Bonjour le monde !';
});
```

Les Fichiers de Routes

Où définir vos routes ?

Les routes sont définies dans le dossier `routes/`

Fichier	Utilisation	Accès
`web.php`	Pages web classiques (HTML)	Sessions, cookies, CSRF
`api.php`	API REST (JSON)	Authentification par token
`console.php`	Commandes Artisan personnalisées	Terminal uniquement

Routes Web (web.php)

```php
// Accès : http://site.com/
Route::get('/', function () {
    return view('welcome');
});
```

Routes API (api.php)

```php
// Accès : http://site.com/api/users
Route::get('/users', function () {
    return User::all();
});
```

2. Les Méthodes HTTP

GET, POST, PUT, DELETE...

Chaque route répond à une **méthode HTTP** spécifique :

Méthode	Utilisation	Exemple
GET	Lire / Afficher des données	Afficher la liste des projets
POST	Créer une nouvelle ressource	Créer un nouveau projet
PUT/PATCH	Modifier une ressource existante	Modifier un projet
DELETE	Supprimer une ressource	Supprimer un projet

Syntaxe Laravel

PHP

```php
// Route GET - Afficher
Route::get('/projets', [ProjetController::class, 'index']);

// Route POST - Créer
Route::post('/projets', [ProjetController::class, 'store']);

// Route PUT - Modifier
Route::put('/projets/{id}', [ProjetController::class, 'update']);

// Route DELETE - Supprimer
Route::delete('/projets/{id}', [ProjetController::class, 'destroy']);
```

Paramètres de Route

Routes dynamiques avec {paramètre}

Les paramètres permettent de capturer des valeurs dans l'URL.

Paramètre obligatoire

```php
// URL : /projets/5
Route::get('/projets/{id}', function ($id) {
    return "Afficher le projet numéro : " . $id;
});

// URL : /utilisateurs/ahmed
Route::get('/utilisateurs/{nom}', function ($nom) {
    return "Bonjour " . $nom;
});
```

Paramètre optionnel

Ajoutez `?` et une valeur par défaut :

```php
// URL : /salut ou /salut/Ahmed
Route::get('/salut/{nom?}', function ($nom = 'Invité') {
    return "Bonjour " . $nom;
});
```

Plusieurs paramètres

Contraintes sur les Paramètres

Valider le format des paramètres

Vous pouvez limiter le format des paramètres avec des expressions régulières :

Méthode where()

```php
// L'id doit être un nombre
Route::get('/projets/{id}', function ($id) {
    // ...
})->where('id', '[0-9]+');

// Le nom doit contenir uniquement des lettres
Route::get('/utilisateurs/{nom}', function ($nom) {
    // ...
})->where('nom', '[A-Za-z]+');
```

Méthodes raccourcies

```php
// whereNumber : uniquement des chiffres
Route::get('/projets/{id}', ...)->whereNumber('id');

// whereAlpha : uniquement des lettres
Route::get('/users/{nom}', ...)->whereAlpha('nom');

// whereAlphaNumeric : lettres et chiffres
Route::get('/posts/{slug}', ...)->whereAlphaNumeric('slug');

// whereUuid : format UUID
Route::get('/orders/{uuid}', ...)->whereUuid('uuid');
```

3. Routes Nommées

Donner un nom à vos routes

Nommer une route permet de l'utiliser facilement dans le code sans écrire l'URL complète.

Définir une route nommée

```php
Route::get('/projets', [ProjetController::class, 'index'])
    ->name('projets.index');

Route::get('/projets/{id}', [ProjetController::class, 'show'])
    ->name('projets.show');

Route::post('/projets', [ProjetController::class, 'store'])
    ->name('projets.store');
```

Utiliser une route nommée

Dans un contrôleur

```php
// Redirection
return redirect()
    ->route('projets.index');

// Avec paramètre
return redirect()
    ->route('projets.show', ['id' => 5]);
```

Groupes de Routes

Organiser les routes avec des préfixes

Les groupes permettent d'appliquer des options communes à plusieurs routes.

Préfixe d'URL

```php
Route::prefix('admin')->group(function () {
    // URL : /admin/dashboard
    Route::get('/dashboard', [AdminController::class, 'dashboard']);

    // URL : /admin/utilisateurs
    Route::get('/utilisateurs', [AdminController::class, 'users']);

    // URL : /admin/parametres
    Route::get('/parametres', [AdminController::class, 'settings']);
});
```

Préfixe de nom

```php
Route::name('admin.')->prefix('admin')->group(function () {
    // Nom : admin.dashboard
    Route::get('/dashboard', ...)->name('dashboard');

    // Nom : admin.users
    Route::get('/utilisateurs', ...)->name('users');
});
```

4. Les Contrôleurs

Organiser la logique de l'application

Définition

Un **contrôleur** est une classe PHP qui regroupe la logique de traitement pour un ensemble de routes liées.

Pourquoi utiliser des contrôleurs ?

❌ Sans contrôleur

```php
// routes/web.php
Route::get('/projets', function () {
    $projets = Projet::all();
    return view('projets.index',
        ['projets' => $projets]);
});

// Code dans le fichier routes !
// Difficile à maintenir...
```

✅ Avec contrôleur

```php
// routes/web.php
Route::get('/projets',
```

Créer un Contrôleur

Avec la commande Artisan

Commande de création

```
$ php artisan make:controller ProjetController
```

Résultat : app/Http/Controllers/ProjetController.php

PHP

```php
<?php

namespace App\Http\Controllers;

class ProjetController extends Controller
{
    // Afficher la liste des projets
    public function index()
    {
        $projets = Projet::all();
        return view('projets.index', ['projets' => $projets]);
    }

    // Afficher un seul projet
    public function show($id)
    {
        $projet = Projet::findOrFail($id);
        return view('projets.show', ['projet' => $projet]);
    }
```

Lier Route et Contrôleur

La syntaxe moderne de Laravel

Importer le contrôleur

```php
// routes/web.php
use App\Http\Controllers\ProjetController;
```

Syntaxe recommandée (Laravel 8+)

```php
// [Classe::class, 'méthode']
Route::get('/projets', [ProjetController::class, 'index']);
Route::get('/projets/create', [ProjetController::class, 'create']);
Route::get('/projets/{id}', [ProjetController::class, 'show']);
Route::post('/projets', [ProjetController::class, 'store']);
```

Ancienne syntaxe (déconseillée)

```php
// Chaîne de caractères - à éviter
Route::get('/projets', 'ProjetController@index');
```

Important

Utilisez toujours la syntaxe avec `::class`. Elle permet à votre IDE de détecter les erreurs et facilite la refactorisation.

Contrôleur de Ressource

CRUD complet en une seule commande

Un contrôleur de ressource génère automatiquement toutes les méthodes CRUD.

Créer un contrôleur ressource

```
$ php artisan make:controller ProjetController --resource
```

Les 7 méthodes générées

Méthode	URL	Action
`index()`	GET /projets	Afficher la liste
`create()`	GET /projets/create	Formulaire de création
`store()`	POST /projets	Enregistrer un nouveau
`show($id)`	GET /projets/{id}	Afficher un seul
`edit($id)`	GET /projets/{id}/edit	Formulaire de modification
`update($id)`	PUT /projets/{id}	Mettre à jour
`destroy($id)`	DELETE /projets/{id}	Supprimer

Route Resource

7 routes en une seule ligne

Déclarer une route ressource

```php
// Une seule ligne crée 7 routes !
Route::resource('projets', ProjetController::class);
```

Vérifier les routes créées

```
$ php artisan route:list
```

Limiter les routes

```php
// Seulement ces méthodes
Route::resource('projets', ProjetController::class)
    ->only(['index', 'show']);

// Toutes sauf celles-ci
Route::resource('projets', ProjetController::class)
    ->except(['destroy']);
```

Route API Resource (sans create/edit)

```php
// Pour les API - 5 routes sans formulaires
```

Injection de Dépendances

Récupérer automatiquement les objets

Laravel peut automatiquement injecter des objets dans vos méthodes de contrôleur.

Injecter la Request

```php
use Illuminate\Http\Request;

public function store(Request $request)
{
    // Accéder aux données du formulaire
    $nom = $request->input('nom');
    $email = $request->input('email');

    // Ou récupérer toutes les données
    $data = $request->all();
}
```

Route Model Binding

Laravel peut récupérer automatiquement un modèle depuis la base de données :

```php
// Route
Route::get('/projets/{projet}', [ProjetController::class, 'show']);

// Contrôleur - $projet est automatiquement récupéré !
public function show(Projet $projet)
{
    // $projet contient déjà le projet avec l'id de l'URL
```

5. Les Middlewares

Filtrer les requêtes HTTP

Définition

Un **middleware** est un filtre qui examine les requêtes HTTP avant qu'elles n'atteignent le contrôleur.

Exemples de middlewares

- **auth** : Vérifie que l'utilisateur est connecté
- **guest** : Vérifie que l'utilisateur n'est PAS connecté
- **verified** : Vérifie que l'email est vérifié
- **throttle** : Limite le nombre de requêtes

Utiliser un middleware

```php
// Sur une route
Route::get('/dashboard', [DashboardController::class, 'index'])
    ->middleware('auth');
```

Exemple Complet : TaskFlow

Organisation des routes du projet

PHP

```php
// routes/web.php
use App\Http\Controllers\ProjetController;
use App\Http\Controllers\TacheController;
use App\Http\Controllers\DashboardController;

// Page d'accueil publique
Route::get('/', function () {
    return view('welcome');
})->name('home');

// Routes protégées (utilisateur connecté)
Route::middleware('auth')->group(function () {

    // Dashboard
    Route::get('/dashboard', [DashboardController::class, 'index'])
        ->name('dashboard');

    // Gestion des projets (CRUD complet)
    Route::resource('projets', ProjetController::class);

    // Tâches d'un projet
    Route::resource('projets.taches', TacheController::class);
});
```

Résultat

Ce code crée automatiquement plus de 14 routes pour gérer les projets et leurs tâches !

Résumé du Chapitre

Routes

Définissent les URLs de l'application

Méthodes HTTP

GET, POST, PUT, DELETE

Contrôleurs

Regroupent la logique métier

Middlewares

Filtrent les requêtes

Commandes à retenir

Commande	Description
`php artisan make:controller NomController`	Créer un contrôleur
`php artisan make:controller NomController --resource`	Contrôleur avec CRUD
`php artisan route:list`	Lister toutes les routes

Prochain chapitre

Chapitre 3 : Vues et Blade - Créer des interfaces utilisateur dynamiques.

Questions ?

Avez-vous des questions sur les routes et les contrôleurs ?

Prochaine séance : TP 2

Création des routes et contrôleurs pour TaskFlow